Práctica 6 · RPC: una llamada remota que parece local

Sistemas Distribuidos · Grado en Tecnologías para la Sociedad de la Información · ETSISI-UPM · 2026-27

Profesor: David Díaz Pardo de Vera · david.diazp@upm.es · Despacho 4307

Dedicación estimada: la clase de laboratorio del martes 27 de octubre y unas 3 h de trabajo personal · Individual

Material: proyecto Maven `sd-prac6-rpc-esqueleto.zip` (Moodle) · IntelliJ IDEA (o VS Code, o Eclipse) · JDK 17 o posterior

(Esta práctica sustituye al enunciado «RPC 1» de cursos anteriores, que se hacía a partir del ejercicio 1 de las transparencias 5-RPC. La plantilla de la parte 1 es la misma, la del repositorio de la asignatura.)

1. Por qué existe esta práctica

En la práctica 4 escribiste, sin llamarlo así, una llamada a procedimiento remoto (ingl. *remote procedure call*, RPC). `BankProxy.deposit(1, 250)` parecía un método Java normal, pero empaquetaba un código de operación y dos argumentos, los enviaba por un socket, esperaba y desempaquetaba la respuesta. Al otro lado, el `switch` de `BankServer` leía el código de operación y llamaba al método que tocaba. A esas dos piezas el middleware les pone nombre: la del cliente es el *stub* o proxy, y la del servidor es el *skeleton* (y el `switch` que elige la operación, el despachador (ingl. *dispatcher*)).

Esta práctica hace tres cosas con esa idea. Primero la aísla: el programa cliente sólo verá una interfaz Java, y no sabrá si detrás hay un objeto local o un proxy que habla por la red. Después le pregunta qué pasa con los errores: una división por cero en el servidor tiene que llegar al cliente como si la división hubiera sido local, y una red caída no tiene equivalente local. Por último, mira qué ocurre cuando el proxy reintenta una llamada que no ha recibido respuesta: según la operación, el reintento es inofensivo o duplica el efecto.

Es el peldaño que lleva del banco de la práctica 4 al ORB de las prácticas 7 a 9. Allí el skeleton tendrá que elegir, además de la operación, el objeto sobre el que se ejecuta.

2. Parte 1 · La plantilla, leída como RPC (unos 15 min)

Abre el proyecto en IntelliJ IDEA (File → Open… sobre la carpeta que contiene `pom.xml`). El paquete `rpc` contiene la plantilla del repositorio de la asignatura: `RPCServer` escucha en el puerto 5000 y, por cada conexión, lee el nombre de un método (una cadena con `readUTF`); si es `sumar`, lee dos enteros, ejecuta la suma y responde con un entero. `RPCClient` hace una llamada a `sumar(5, 3)`.

Ejecuta `RPCServer` y después `RPCClient`. Debe imprimir `Resultado de la suma: 8`.

En esta plantilla no hay proxy ni skeleton separados: el cliente escribe los bytes en su `main` y el servidor los lee en `RPCHandler.run()`. Señala en el código las líneas que harían de proxy y las que harían de skeleton (contéstalo en tu `RESPUESTAS.md`, junto a P1).

Experimento E1. Cambia en el cliente el nombre del método a `"restar"` (sólo la cadena; el servidor no lo conoce) y ejecútalo varias veces contra el mismo servidor. Unas veces imprime un número; otras, el cliente termina con una `SocketException`.

P1. ¿Qué número imprime el cliente? Explícalo a partir de los bytes que escribe el servidor en la rama `else` y de los que lee el cliente. (La cadena del mensaje mide 29 caracteres, todos ASCII.) Y la excepción de las otras veces: fíjate en qué bytes del cliente no llega a leer nunca el servidor antes de cerrar. Por último, ¿qué tendría que llevar la respuesta para que el cliente supiera distinguir un resultado de un error?

3. Parte 2 · Un stub y un skeleton de verdad (unos 45 min)

El paquete `calculadora` reorganiza la misma idea en piezas con nombre. La tabla 1 dice quién hace qué, y la figura 1 sigue una llamada de principio a fin.

Clase	Papel	Estado en el esqueleto
`Calculadora`	la interfaz del servicio: `sumar`, `restar`, `multiplicar`, `dividir`, `acumular`	completa
`CalculadoraLocal`	la implementación de verdad, el «sirviente» (ingl. *servant*), sin nada de red	completa
`Protocolo`	los códigos de operación, los dos estados de respuesta, el puerto	completa
`CalculadoraProxy`	el stub del cliente: implementa `Calculadora` hablando por la red	TODO (1) y (2)
`CalculadoraSkeleton`	el skeleton del servidor: lee la petición y llama al sirviente	TODO (3), (4) y (5)
`ServidorCalculadora`	el bucle de `accept()`, con un hilo por conexión	completa
`DemoCalculadora`	el programa cliente: `local` o `remota`	completa
`DemoAcumular`	el programa de la parte 3	completa

Tabla 1. Las piezas de la calculadora remota. El programa cliente sólo conoce `Calculadora`.

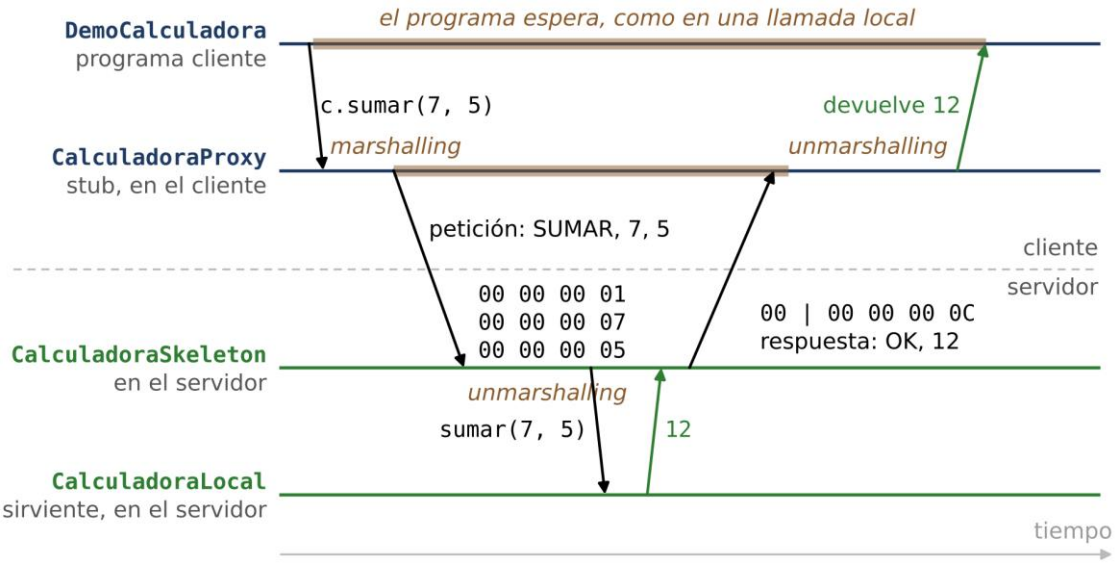


Figura 1. Una llamada remota, `sumar(7, 5)`, a través de las cuatro piezas. Las barras marrones son esperas: el programa y el proxy se quedan bloqueados hasta que llega la respuesta, igual que esperarían a una llamada local.

3.1 El protocolo

Una conexión TCP por llamada: el proxy abre un socket en cada método, envía la petición, lee la respuesta y lo cierra. Es el modelo de la plantilla y el de la práctica 7; el banco de la práctica 4 usaba una conexión por sesión.

Sentido	Contenido, en orden
Cliente → servidor	`int` opcode, y después los argumentos, todos `int` (dos en `sumar`, `restar`, `multiplicar` y `dividir`; uno en `acumular`)
Servidor → cliente, si todo va bien	`byte` estado = 0 (OK), `int` resultado
Servidor → cliente, si la operación falla	`byte` estado = 1 (ERROR), UTF mensaje

Los opcodes están en `Protocolo`: 1 sumar, 2 restar, 3 multiplicar, 4 dividir, 5 acumular.

3.2 Tus tareas

`sumar` funciona de extremo a extremo y sirve de modelo. Completa, por orden:

1. **TODO (1)**, en el proxy: `restar`, `multiplicar`, `dividir` y `acumular`, como `sumar`. Todos delegan en el método privado `invocar`, que reintenta, y éste en `unaVez`, que hace el marshalling de la petición.
2. **TODO (3)**, en el skeleton: los cuatro casos que faltan en el `switch` de `ejecutar`.
3. **TODO (4)**, en el skeleton: cuando el sirviente lanza una `ArithmeticException`, la excepción no se envía. Devuelve una `Respuesta` con estado `ERROR` y el mensaje de la excepción.
4. **TODO (5)**, en el skeleton: con estado `ERROR`, escribe el mensaje con `writeUTF`.
5. **TODO (2)**, en el proxy: con estado `ERROR`, lee el mensaje y lanza una `ArithmeticException` con ese mensaje.

Ejecuta `DemoCalculadora` con el argumento `local`. Después arranca `ServidorCalculadora` y ejecuta `DemoCalculadora` con el argumento `remota`. Las dos salidas deben coincidir línea a línea (salvo la primera, que dice cuál es):

```
sumar(7, 5)       = 12
restar(7, 5)      = 2
multiplicar(7, 5) = 35
dividir(7, 5)     = 1
dividir(7, 0)     -> ArithmeticException: / by zero
```

P2. En remoto, la división por cero ocurre en el servidor y el cliente recibe una `ArithmeticException` con el mismo mensaje. ¿Ha viajado la excepción por la red? Di qué bytes viajan y en qué clase, y en qué máquina, se construye la excepción que ve el programa cliente.

P3. Detén el servidor y ejecuta otra vez la demo en remoto. ¿Qué excepción aparece, y en qué línea de la demo? Esa excepción no puede ocurrir con `CalculadoraLocal`. `LlamadaRemotaException` hereda de `RuntimeException`, así que el compilador no obliga a tratarla; en Java RMI su equivalente, `java.rmi.RemoteException`, es obligatorio declararlo en cada método remoto. ¿Qué gana el programador del cliente con esa obligación, y qué pierde la «transparencia» de la llamada?

4. Parte 3 · Reintentar no siempre es inofensivo

(unos 20 min, en casa si no da tiempo en clase)

El proxy tiene un plazo de espera de 2 s (`setSoTimeout`, como el de 3 s de la práctica 4) y puede reintentar: el tercer argumento de su constructor es el número de intentos. Si la conexión falla, se corta o la respuesta no llega a tiempo, `invocar` vuelve a enviar la misma petición. El proxy no puede saber si el servidor llegó a ejecutarla.

`acumular(x)` es la tecla M+ de una calculadora: suma `x` a una memoria que guarda el servidor y devuelve el total nuevo. Las otras cuatro operaciones no guardan nada.

El modelo del experimento. Arranca `ServidorCalculadora` con el argumento `--pierde-respuestas`. Con esa opción, el servidor ejecuta todas las peticiones, pero de cada dos que recibe (la primera, la tercera, la quinta…) cierra la conexión sin enviar la respuesta: la operación se ha hecho y el cliente no se entera. El proxy lo nota en cuanto intenta leer (el flujo se acaba), sin esperar a que venza el plazo. La red, el cliente y el resto del servidor funcionan bien, y nadie se cae. `DemoAcumular`, contra ese servidor recién arrancado, llama cinco veces a `acumular(10)` con un proxy de 3 intentos, una llamada detrás de otra, sin más clientes.

P4. Al terminar las cinco llamadas, ¿qué total esperaría quien no supiera nada de la red, y cuál imprime `DemoAcumular`? Explica la diferencia dibujando el diagrama de tiempos de la primera llamada (cliente y servidor, con el tiempo de izquierda a derecha, como en el Tema 2): cada petición, cada respuesta o su ausencia, y el valor de la memoria del servidor después de cada ejecución. ¿Cambiaría el resultado si las cinco llamadas fueran a `sumar(10, 0)`? ¿Y si el proxy tuviera un solo intento? Di qué ve el programa en la primera llamada y qué ha pasado en el servidor.

Lo que has observado tiene nombre, y ya lo viste en el Tema 2 con los enlaces. Una llamada que se reintenta hasta recibir respuesta da entrega **al menos una vez** (ingl. *at-least-once*): si la respuesta llega, la operación se ha ejecutado una vez o más. Basta cuando la operación es **idempotente** (ingl. *idempotent*), es decir, cuando ejecutarla varias veces deja el mismo efecto que ejecutarla una. Sin reintentos, la entrega es **como mucho una vez** (ingl. *at-most-once*): si la respuesta no llega, la operación se ha ejecutado una vez o ninguna, y el cliente no sabe cuál. La parte 4 añade lo que falta para que reintentar sea seguro siempre.

5. Parte 4 (opcional) · Reintentar sin repetir

Para que reintentar sea seguro también con `acumular`, el servidor tiene que reconocer una petición repetida y no volver a ejecutarla: le basta con reenviar la respuesta que ya calculó. Para eso cada llamada lleva un identificador, el mismo en todos sus reintentos.

El proxy ya lo envía cuando `Protocolo.CON_ID` vale `true`: un `long` delante del opcode. Pon esa constante a `true` (es parte del protocolo, así que la ven los dos extremos), vuelve a compilar, arranca de nuevo el servidor y completa el **TODO (6)** del skeleton: si el identificador ya está en el mapa `hechas`, lee los argumentos sin usarlos (`consumirArgumentos`) y reenvía la respuesta guardada; si no está, ejecuta la operación y guarda su respuesta. Repite el experimento de la parte 3: ahora el total debe ser 50.

P5. ¿Qué guarda tu servidor por cada llamada, y hasta cuándo tendría que guardarlo? Imagina que la memoria de `acumular` se guarda en disco y sobrevive a un reinicio, pero el mapa no: ¿qué pasaría con una llamada ya ejecutada cuyo reintento llega después del reinicio? (Compara con el enlace perfecto del Tema 2, que elimina los duplicados del enlace obstinado.)

Con reintentos, filtrado de duplicados y respuestas guardadas, toda llamada que recibe respuesta se ha ejecutado exactamente una vez, siempre que sus reintentos lleguen uno detrás de otro, como aquí; si el proxy reintentara hasta tener respuesta, el efecto sería el de la entrega **exactamente una vez** (ingl. *exactly-once*) del Tema 2, que el enlace perfecto consigue de la misma manera.

Ojo con los nombres si lees Coulouris y otros (5.ª ed., §5.3.1, figura 5.9): el libro llama *maybe* a la llamada sin reintentos y *at-most-once* a la combinación de esta parte, con reintentos y filtro de duplicados. Las ideas son las mismas; la asignatura usa los nombres de Kleppmann, que son los del glosario.

6. Lo que este peldaño te da

Para el examen de laboratorio PR-EX-2 («RMI. ORB.» en la guía de aprendizaje) y para las prácticas 7 a 9: reconocer el proxy, el skeleton y el despachador en un código que no los nombra; escribir el marshalling de una petición y de una respuesta con estado; explicar qué viaja cuando una operación remota falla; y decidir qué garantía da una llamada según el proxy reintente o no y el servidor filtre o no los duplicados.

7. Entregables

Un zip de tu proyecto Maven con las partes 2 y 3 (y la 4, si la haces), y un breve `RESPUESTAS.md` en su raíz con P1 a P4 (y P5, si haces la parte 4): unas líneas cada una, razonadas. El diagrama de P4 puede ir como foto o como dibujo en texto.

Sube el zip a la tarea de Moodle antes del **domingo 8 de noviembre a las 23:59**. La entrega es voluntaria y no lleva nota: sirve para que el profesor te la corrija. El trabajo es individual; discutir ideas está bien, compartir código no.

Notas al pie

Las palabras. En el cliente, *stub* y proxy son lo mismo; en el servidor, *skeleton* (esqueleto). Java RMI llama *stub* a la pieza del cliente y la genera sola, al ejecutar, a partir de una interfaz que extiende `java.rmi.Remote`; en el servidor no hay que escribir nada parecido a un skeleton. gRPC las genera a partir de un fichero `.proto`: el *stub* del cliente y una clase base del servidor (`…ImplBase`) que tú extiendes con el código de verdad, como `CalculadoraLocal`. Lo verás en la práctica 10.

Para leer más. Coulouris, Dollimore, Kindberg y Blair, *Distributed Systems: Concepts and Design*, 5.ª ed., §5.3 («Remote procedure call»), en particular §5.3.1 (interfaces, semántica de llamada y transparencia) y §5.3.2 (la implementación, con el stub y el despachador). Kleppmann, *Distributed Systems* (notas del curso de Cambridge), §1.3 y §5.1. Hernández-Peñaloza y Serrano (2024); Castilla e Iglesias (2025): 5-RPC, transp. 10 a 13 y 21 (en Moodle, «Materiales de cursos anteriores»).